Practical : 6	Solve a constraint satisfaction problem such as map coloring, Sudoku or N-Queens using backtracking and consistency checking.
Date : 28/09/26	

Aim: -

To solve a Constraint Satisfaction Problem (CSP) using backtracking and consistency checking.

Algorithm:-

Map Coloring Using Backtracking

1. Start the program.
2. Define the map regions and their connections.
3. Initialize the available colors as Red, Green, and Blue.
4. Check whether assigning a color to a region satisfies the constraints.
5. Select an uncolored region and try an available color.
6. If the color is consistent, assign it and move to the next region.
7. If no color works, backtrack by removing the previous assignment and try another color.
8. Repeat until all regions are colored or no solution is possible.
9. Display the final color assignment.
10. Stop the program.

Program:-

```
# AI Practical 6: Map Coloring Using Backtracking

# Step 1: Define the map and connections
map_data = {
    'A': ['B', 'C'],
    'B': ['A', 'C', 'D'],
    'C': ['A', 'B', 'D'],
    'D': ['B', 'C']
}

# Step 2: Define available colors
colors = ["red", "green", "blue"]


# Step 3: Check consistency
def is_consistent(region, color, assignment):
    for neighbor in map_data[region]:
        if neighbor in assignment and assignment[neighbor] == color:
            return False
    return True


# Step 4: Solve using backtracking
def solve_map(assignment):

    # If all regions are colored, return the solution
    if len(assignment) == len(map_data):
        return assignment

    # Find the next uncolored region
    for region in map_data:
        if region not in assignment:
            break

    # Try each available color
```

```
    for color in colors:
        if is_consistent(region, color, assignment):
            assignment[region] = color

            result = solve_map(assignment)

            if result is not None:
                return result

            # Backtrack
            del assignment[region]

    return None


# Step 5: Display the solution
solution = solve_map({})

if solution is not None:
    print("Map Coloring Solution:")
    for region, color in solution.items():
        print(region, "=", color)
else:
    print("No solution exists.")
```

Output:-

```
Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Vishnu\OneDrive>python AI_Practical_6.py
Map Coloring Solution:
A = red
B = green
C = blue
D = red
```

Conclusion:-

Thus, the Map Coloring Problem was successfully solved using backtracking and consistency checking in Python. The program assigns colors to regions while ensuring that no two connected regions have the same color.

.